

MAKAR

DISTRIBUTED CARGO FORENSICS, PROVENANCE & RECONSTRUCTION SYSTEM

System Architectural Specification & Technical Demonstration Report

Team: Vanar Sena

Author: Ariet Jha

Architecture • Detection engines • Evidence fusion • Reconstruction • Benchmarks • Demonstration walkthrough

Context & Strategic Mandate

In global maritime trade logistics, cargo manifest tampering poses an existential threat to regulatory compliance, customs enforcement, and international supply chain security. Malicious actors systematically alter cargo weights, substitute item descriptions, omit transshipment legs, and manipulate container ownership histories to facilitate contraband smuggling, tariff evasion, and trade-based money laundering. Detecting these illicit modifications is fundamentally challenging because operational shipping networks operate under a complete absence of ground-truth labels. Furthermore, false alarms incur prohibitive financial and operational costs: an improperly flagged shipment manifest stalls a legitimate, multi-million-dollar cargo consignment at a port terminal, triggering severe supply chain compounding bottlenecks, vessel delay charges, and demurrage penalties. The strategic imperative for the Makar platform is to deliver high-precision anomaly recall while maintaining zero tolerance for false alarms on legitimate shipments, yielding fully explainable, court-admissible forensic output.

Deconstruct Core Forensic Objectives

The platform operational mandate is structured across four distinct forensic phases. The following matrix delineates the mandate, target output state, and governing operational constraints for each phase:

Phase	Operational Mandate	Target Output State	Strict Constraints
Detect	Identify inconsistent, impossible, or anomalous manifest entries across spatial, temporal, graph, and physical dimensions.	Raw anomaly observations accompanied by per-engine severity scores $s_i \in [0, 1]$.	Engines emit raw findings only; no detection engine may independently assign a final verdict or classify a record.
Identify	Classify specific tampering vectors (modification, deletion, duplication, fabrication) and assign calibrated probabilities.	Categorized tampering classification paired with a signed log-odds posterior probability score P .	Must synthesize cross-layer evidence fusion and conflict arbitration; single uncorroborated features cannot convict.
Reconstruct	Restore tampered or missing manifest entries to their true historical state using physical and contextual invariants.	Candidate original values paired with multi-criterion evaluation scores and confidence metrics.	Output states must be explicitly classified; silent field overwrites or ungrounded parameter guessing are strictly prohibited.
Report	Construct an auditable, court-admissible forensic evidence chain detailing the step-by-step reasoning path.	Structurally complete Suspicious Activity Reports (SAR) and interactive visual investigation interfaces.	Reasoning must be derived strictly from deterministic arithmetic and cryptographically verifiable evidentiary traces.

Evaluate Output Disposition Classifications

To guarantee total operational transparency and eliminate silent record state mutations, Makar enforces four mandatory, non-negotiable disposition state classifications for every output record:

- **ORIGINAL:** Verified authentic records demonstrating complete structural, physical, and cryptographic integrity without evidentiary suspicion.
- **REPAIRED:** Compromised records where the candidate reconstruction engine has restored true historical parameters above the designated confidence threshold ($\text{Score}(C_k) \geq 0.85$).
- **REMOVED:** Fabricated entries or structural duplicates lacking underlying physical lineage that are purged from the authoritative manifest view.
- **UNRECOVERABLE:** Confirmed tampered records where contextual evidence is insufficient to generate a high-confidence repair hypothesis, requiring human forensic intervention.

Analyze the Foundational Processing Pipeline

Makar operates under an absolute architectural doctrine governing data abstraction and state evaluation:

$\text{Observation} \rightarrow \text{Evidence} \rightarrow \text{Inference} \rightarrow \text{Decision}$

Under this multi-tiered architecture, independent detection modules strictly process data observations and emit structured evidence objects containing severities $s_i \in [0, 1]$. Detectors are completely decoupled from record classification verdicts. The confidence layer converts aggregated evidence into an inference (calibrated posterior probability), while the candidate reconstruction layer converts that inference into an operational decision (disposition assignment).

Decoupling detection from final classification ensures system extensibility: evidence weights can be recalibrated, individual engine layers can be ablated, and evidentiary blame can be asymmetric redistributed between contradictory record pairs without requiring code modifications within the underlying detection engines.

Assess Exculpatory Messiness & Noise Immunity

Real-world maritime export data is inherently scruffy. Heterogeneous software systems emit inconsistent timestamp representations, localized character encodings, and missing optional attributes. A foundational doctrine of Makar is that benign data entry noise must be explicitly treated as exculpatory evidence of a sloppy human export rather than malicious tampering.

To achieve this, Makar assigns FORMAT evidence findings a negative fusion weight (-0.90). By transforming benign untidiness into an exculpatory signal, a scruffy, legitimate record scores as less suspicious than a polished, synthetic forgery. Across empirical evaluations, normalization modules identified 255 FORMAT findings against 291 injected noise entries with zero false alarms on clean or noisy records across all evaluation seeds (Seed 481516, Seed 271828, and Seed 999331), establishing a 0.00% false-alarm rate on noise-only records.

Section Transition

Having established Makar’s foundational philosophy and state classifications, the following section details the structural design of the end-to-end processing pipeline and ingestion normalizer.

2. End-to-End System Architecture & Ingestion Pipeline

Context & Strategic Mandate

The Makar platform is architected as a modular, isolation-bounded pipeline designed to ingest untrusted, highly variable cargo manifests and transform them into tamper-evident forensic inferences. Raw, un-normalized record streams pass through canonical transformation stages, graph context building, parallel forensic detection, conflict arbitration, log-odds fusion, and candidate reconstruction to produce auditable output states.

Analyze Input Manifest Normalization

Makar’s normalization parser ingests non-canonical text, non-standard date variants, and irregular numeric representations, emitting typed internal domain models alongside explicit FORMAT evidence findings.

Table 1: Supported Timestamp Variants & Mapping

Incoming Format Variant	Example Raw Input	Canonical Output Mapping	Raised Evidence Code
ISO 8601 with Z	2026-01-03T04:00:00Z	2026-01-03 04:00:00+00:00	None (Canonical Standard)
ISO 8601 without Zone	2026-01-03T04:00:00	2026-01-03 04:00:00+00:00	TIMESTAMP_FORMAT_VARIANT
Space Separator	2026-01-03 04:00:00	2026-01-03 04:00:00+00:00	TIMESTAMP_FORMAT_VARIANT
Sub-second Precision	2026-01-03T04:00:00.000Z	2026-01-03 04:00:00+00:00	TIMESTAMP_FORMAT_VARIANT
Explicit ISO Offset	2026-01-03T04:00:00+00:00	2026-01-03 04:00:00+00:00	TIMESTAMP_FORMAT_VARIANT
Month-First Slashes	01/03/2026 04:00:00	2026-01-03 04:00:00+00:00	TIMESTAMP_FORMAT_VARIANT
Day-First Dashes	03-01-2026 04:00:00	2026-01-03 04:00:00+00:00	TIMESTAMP_FORMAT_VARIANT
Epoch Seconds	1767412800	2026-01-03 04:00:00+00:00	TIMESTAMP_FORMAT_VARIANT

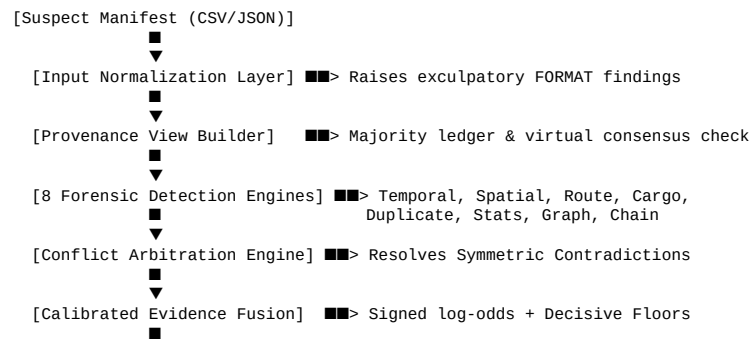
Table 2: Benign Blank Tokens & Field Mapping

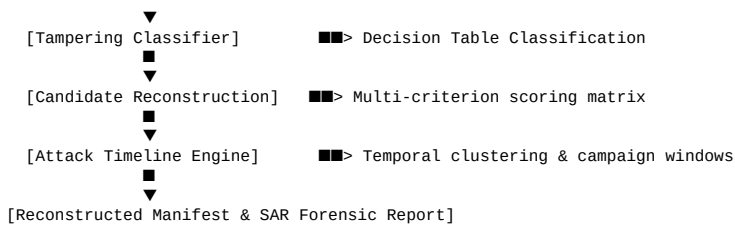
Token Pattern Class	Recognized Blank Inputs	Internal Mapping	Raised Evidence Code
Empty / Hyphenated	"" , -, --	None (Absent)	FIELD_BLANK (Exculpatory)
Null Literals	NULL, null, nil, nan	None (Absent)	FIELD_BLANK (Exculpatory)
Unspecified Markers	?, N/A, unknown, unspecified, tbd, none	None (Absent)	FIELD_BLANK (Exculpatory)

Crucially, blank tokens resolve to an explicit absent field state (None) rather than an "unknown entity". Resolving vessel_id = "N/A" as absent prevents the normalizer from raising spurious "unknown entity reference" anomalies, which would otherwise flag empty cells as malicious edits.

Map Full System Processing Flow

Data progresses sequentially through the modular Makar framework:





Evaluate System Separation of Concerns

Makar enforces strict operational boundaries between reference world data and ground-truth injection logs:

- World Model (world.json): Master reference dataset describing valid maritime domain entities, including 18 ports, 26 vessels, 24 geography-constrained trade routes, 20 cargo owners, active shipments, and registered containers. This represents domain knowledge accessible to a legitimate maritime operator.
- Private Injection Log (ground_truth.json): The isolated audit log containing exact parameters of injected corruptions and noise. Core analytical modules within core/ never import, read, or query this file. It is consumed exclusively by evaluation/ modules during scoring.

Section Transition

With the architecture and ingestion pipeline established, the next section details the mechanics of Makar's eight parallel forensic engines and its 49 evidence codes.

3. The Eight Forensic Engines & 49 Evidence Codes

Context & Strategic Mandate

Makar's analytical core relies on eight independent, domain-specific forensic detection engines. Each engine evaluates orthogonal physical, mathematical, temporal, spatial, and cryptographic invariants, emitting structured evidence codes without executing premature classification verdicts.

Deconstruct the Eight Forensic Engines

1. Temporal Engine

- Detection Domain: Evaluates chronological sequencing, dwell-time distributions, and lifecycle ordering.
- Underlying Mechanism: Parses arrival, departure, and event timestamps across (container, shipment) scopes.
- Operational Logic: Enforces event lifecycle ordering (CREATED $\rightarrow$ LOADED $\rightarrow$ DEPARTED $\rightarrow$ ARRIVED $\rightarrow$ TRANSFERRED $\rightarrow$ UNLOADED $\rightarrow$ DELIVERED $\rightarrow$ CANCELLED). Identifies overlapping port stays (SIMULTANEOUS_PRESENCE) and flags future-dated event timestamps against the analysis clock while excluding scheduled departures (departure_timestamp).

2. Geospatial Engine

- Detection Domain: Evaluates spatial feasibility, vessel velocity envelopes, and coordinate boundaries.
- Underlying Mechanism: Computes maritime passage distance between port coordinates using Haversine geometry multiplied by a 1.25 $\times$ sea-distance factor to reflect actual routing channel passage over great-circle arcs.
- Operational Logic: Calculates required transit speed $v_{\text{req}} = d_{\text{sea}} / \Delta t$. Evaluates v_{req} against vessel operational envelopes (cruise_speed_knots, max_speed_knots). Emits SPEED_INFEASIBLE when $v_{\text{req}} > v_{\text{max}}$.

3. Route Engine

- Detection Domain: Validates sequence alignment against geography-constrained trade corridors.
- Underlying Mechanism: Maps reported port calls against the ordered port_sequence defined in the world model for the assigned route_id.
- Operational Logic: Flags off-route port calls (OFF_ROUTE_PORT), out-of-order calls (ROUTE_SEQUENCE_BREAK), and declared destination discrepancies where the final port does not match the route terminus (DESTINATION_CONTRADICTION).

4. Cargo Conservation Engine

- Detection Domain: Enforces physical mass, volume, declared value, and ownership continuity invariants.

- Underlying Mechanism: Tracks container state transitions across consecutive port calls.
- Operational Logic: Evaluates mass transitions: $\text{Weight}_{\text{out}} = \text{Weight}_{\text{in}} + \text{Mass}_{\text{loaded}} - \text{Mass}_{\text{unloaded}}$. Unexplained mutations without a cargo-modifying event (LOADED, UNLOADED, TRANSFERRED) raise WEIGHT_NOT_CONSERVED. Handles splittable cargo classes (e.g., partial discharges of TEXTILES or CEMENT at transshipment hubs requiring an accompanying UNLOADED event).

5. Duplicate Detection Engine

- Detection Domain: Identifies exact, structural, and near-duplicate record fabrications.
- Underlying Mechanism: Implements a three-tier matching pipeline paired with a time-windowed candidate blocking strategy.
- Operational Logic: Candidates are blocked on container_id or (owner, cargo_type) within a rolling temporal window, suppressing $O(n^2)$ search spaces down to $O(n)$ near-linear execution time. Matches exact payload hashes (EXACT_DUPLICATE), identical business attributes with altered record IDs (STRUCTURAL_DUPLICATE), or fuzzy field similarity metrics (FUZZY_DUPLICATE).

6. Statistical Engine

- Detection Domain: Identifies multi-variate and univariate distributional anomalies.
- Underlying Mechanism: Operates non-parametric robust estimators (Median/MAD, IQR) and unsupervised algorithms (Isolation Forest, Local Outlier Factor, DBSCAN) scoped per cargo class.
- Operational Logic: Evaluates container gross weight and value density (USD/kg) against peer groups. Capped at a maximum severity of ≤ 0.55 , ensuring statistical findings act strictly as corroborative signals and can never convict independently.

7. Graph/Lineage Engine

- Detection Domain: Analyzes relationship topologies and missing event node sequences.
- Underlying Mechanism: Constructs a directed cargo intelligence graph using NetworkX representing entity dependencies.
- Operational Logic: Traverses graph paths to detect broken lineage chains (LINEAGE_BREAK), unreferenced entities (UNKNOWN_ENTITY_REFERENCE), and orphan records lacking valid relationships (ORPHAN_RECORD).

8. Provenance Engine

- Detection Domain: Audits cryptographic block headers and cross-node consensus state roots.
- Underlying Mechanism: Re-evaluates Merkle tree root assertions across virtual distributed ledger nodes.
- Operational Logic: Compares record content hashes against committed block headers to detect post-hoc database alterations (RECORD_HASH_MISMATCH) and missing history commitments (RECORD_NOT_IN_CHAIN).

Synthesize the 10 Reasoning Layers & 49 Evidence Codes

The framework aggregates 49 distinct evidence codes across 10 reasoning layers, applying explicit weightings and severity multipliers:

Evidence Layer	Fusion Weight	Active Evidence Codes	Mechanics, Multipliers & Severity Caps
BLOCKCHAIN	4.50	RECORD_HASH_MISMATCH, RECORD_NOT_IN_CHAIN, NODE_STATE_DIVERGENCE, BLOCK_CHAIN_BROKEN	Cryptographic interlock. RECORD_HASH_MISMATCH is downgraded to low severity if an export field is missing.
TEMPORAL	2.20	SIMULTANEOUS_PRESENCE, REVERSE_CHRONOLOGY, IMPOSSIBLE_TRANSIT, FUTURE_EVENT, EVENT_ORDER_VIOLATION, DWELL_TIME_ANOMALY, TIMESTAMP_MISSING, SEQUENCE_GAP	Timeline integrity checks. DWELL_TIME_ANOMALY applies a $\times 0.35$ severity multiplier; SEQUENCE_GAP applies $\times 0.40$.
DUPLICATE	2.20	EXACT_DUPLICATE, STRUCTURAL_DUPLICATE, FUZZY_DUPLICATE	Redundancy checks. Time-windowed candidate blocking strategy maintains near-linear performance.
SPATIAL	2.00	SPEED_INFEASIBLE, COORDINATE_PORT_MISMATCH, COORDINATE_OUT_OF_RANGE	Velocity and coordinate bounds. COORDINATE_PORT_MISMATCH applies an $\times 0.80$ severity multiplier.
ROUTE	2.00	OFF_ROUTE_PORT, ROUTE_SEQUENCE_BREAK, DESTINATION_CONTRADICTION, UNKNOWN_ROUTE	Geography-constrained trade corridor alignment checks.

Evidence Layer	Fusion Weight	Active Evidence Codes	Mechanics, Multipliers & Severity Caps
CARGO	1.95	WEIGHT_NOT_CONSERVED, CARGO_DRIFT_UNEXPLAINED, VALUE_NOT_CONSERVED, CONTAINER_COUNT_NOT_CONSERVED, OWNER_CHANGED_WITHOUT_TRANSFER, CARGO_TYPE_MUTATED, WEIGHT_EXCEEDS_CAPACITY	Conservation law validation. CARGO_DRIFT_UNEXPLAINED evaluates multi-step cumulative drift.
GRAPH	1.85	ORPHAN_RECORD, LINEAGE_BREAK, MISSING_EXPECTED_EVENT, GRAPH_CONFLICT, UNKNOWN_ENTITY_REFERENCE	Topological graph continuity. LINEAGE_BREAK and MISSING_EXPECTED_EVENT apply a $\times 0.70$ multiplier.
IDENTITY	1.20	UNKNOWN_OWNER, OWNER_ALIAS_COLLISION, ID_FORMAT_INVALID	Entity resolution and owner naming alias collision detection.
STATISTICAL	0.80	PEER_GROUP_OUTLIER, ROBUST_Z_OUTLIER, IQR_OUTLIER, ISOLATION_FOREST_OUTLIER, LOF_OUTLIER, DBSCAN_NOISE, DISTRIBUTION_SHIFT	Low-weight layer capped at ≤ 0.55 . Multipliers: IQR_OUTLIER ($\times 0.60$), LOF_OUTLIER ($\times 0.70$), DBSCAN_NOISE ($\times 0.50$). Cannot convict independently.
FORMAT	-0.90	FIELD_BLANK, TIMESTAMP_FORMAT_VARIANT, NAME_UNNORMALISED, NUMERIC_FORMAT_VARIANT, CASE_INCONSISTENCY	Negative fusion weight. Treats export formatting untidiness as exculpatory evidence of benign human data entry.

Explain Multi-Layered Corroboration

Makar rejects single-feature conviction. High-confidence fraud identification requires cross-layer evidence accumulation. For example, a minor statistical weight deviation (PEER_GROUP_OUTLIER, severity 0.45) combined with an extended port stay (DWELL_TIME_ANOMALY, severity 0.35) yields a fused posterior probability far below the suspicion threshold. However, if those signals coincide with a cargo conservation failure (WEIGHT_NOT_CONSERVED, severity 0.85) and an off-route call (OFF_ROUTE_PORT, severity 0.90), their fused log-odds drive the posterior tampering probability above 0.95. This multi-layered design prevents low-level anomalies from triggering false alarms while ensuring complex attacks are caught.

Section Transition

The structural rules governing evidence collection transition directly into the mathematical formulations used to establish baselines and fuse probabilities.

4. Mathematical Foundations & Statistical Baselines

Context & Strategic Mandate

Operating in unlabelled maritime environments requires learning baseline norms directly from contaminated manifest feeds. Because incoming datasets contain tampered entries, statistical baseline estimation must utilize robust mathematical estimators capable of resisting severe outlier distortion.

Evaluate Robust Baseline Learning

Parametric estimators like mean (μ) and standard deviation (σ) break down when exposed to single extreme outliers. For instance, in a sample of 20 container records with a true mass of 10,000 kg, a single tampered entry with a $7\times$ weight inflation (70,000 kg) shifts the sample mean to 13,000 kg and inflates the standard deviation from 500 kg to over 13,000 kg. This distortion masks subtle modifications on adjacent records.

Makar resolves this by operating Median ($\tilde{x}$) and Median Absolute Deviation (MAD) as its primary statistical baseline estimators:

$$\text{MAD} = \text{median}(|x_i - \text{median}(X)|)$$

Median and MAD maintain a 50% breakdown point, meaning clean statistical baselines are recovered even if up to half the records in a cargo group are corrupted.

When sample populations exhibit zero scale—such as uniform container_count fields where over 50% of records share identical modal values—MAD evaluates to zero. Dividing by zero would cause standard scoring functions to assign infinite outlier scores to legitimate non-modal entries. Makar implements a robust fallback execution chain:

$$\text{MAD} \rightarrow \text{IQR (Interquartile Range)} \rightarrow \text{Standard Deviation}$$

If MAD evaluates to zero, the engine falls back to IQR ($\text{IQR} = Q_3 - Q_1$). If IQR also evaluates to zero, it falls back to sample standard deviation with a floating-point $\epsilon = 10^{-6}$ floor, preventing divide-by-zero instability.

Deconstruct Probability Math & Calibrated Log-Odds Fusion

Makar aggregates findings across all ten evidence layers into a calibrated posterior tampering probability P using signed log-odds arithmetic:

$$P = \frac{1}{1 + e^{-L}} \quad \text{where} \quad L = L_0 + \sum_{i=1}^K (w_i \cdot s_i)$$

- L_0 : Prior log-odds, set to -2.94, corresponding to a 5% baseline prior tampering probability ($P_{\text{prior}} = 0.05$).
- w_i : Designated fusion weight for evidence layer i (e.g., +2.20 for TEMPORAL, -0.90 for FORMAT).
- s_i : Engine severity score ($s_i \in [0, 1]$).

Analyze Decisive Impossibility Floors

Standard log-odds arithmetic exhibits a key limitation: high prior resistance ($L_0 = -2.94$) can prevent a single physical impossibility (e.g., a container appearing in two distant ports simultaneously) from reaching the conviction threshold without excessively inflating global layer weights. However, raising layer weights globally causes low-severity anomalies (like DWELL_TIME_ANOMALY) to trigger false alarms.

Makar resolves this by implementing Decisive Impossibility Floors (fusion.decisive_floors). When a high-confidence logical or physical impossibility code fires above its severity threshold, the final fused probability is floored at a deterministic level ($P \geq 0.95$), bypassing standard log-odds addition:

Fusion Strategy Configuration	Precision	Recall	F1-Score	Clean Record False Positives
Original Base Weights	0.991	0.897	0.942	2
Raised Layer Weights (Without Floors)	0.965	0.910	0.937	7
Original Base Weights + Decisive Floors	0.991	0.910	0.949	2

Lifting base layer weights increased false positives on clean records from 2 to 7. Combining original base weights with decisive impossibility floors achieved the highest F1-score (0.949) while preserving false-alarm discipline.

Section Transition

Mathematical fusion converts raw findings into posterior probabilities. However, physical contradictions inherently involve pairs of records, requiring an arbitration mechanism to assign blame correctly.

5. Conflict Arbitration & Asymmetric Blame Attribution

Context & Strategic Mandate

Physical contradictions—such as a container reported in two ports simultaneously or a transit leg requiring impossible vessel speed—involve two related records. Standard detection engines flag both records identically, creating the Symmetric Contradiction Problem, where innocent records are flagged alongside tampered ones.

[Symmetric Contradiction Detected]

/\ Record A (Legitimate) Record B (Tampered) - On Declared Route - Off-Route Call - Verified Hash on Chain - Hash Mismatch on Chain - Clustered Port Call - Isolated Event Node - Clean Timeline Neighbors - Corrupted Timeline \ / [Multi-Signal Corroboration Evaluator] /\ Severity Damped ($\gamma \leq 0.10$) Full Penalty Retained ($s_i \geq 0.85$) Fused Prob < 0.20 Fused Prob > 0.95 Disposition: ORIGINAL Disposition: MODIFIED

Deconstruct the Corroboration Engine

Makar solves this by decoupling observation from blame assignment. When a contradiction is detected, evidence is initially attached to both records. Before final fusion, the Conflict Arbitration Engine evaluates five independent corroboration signals to determine which record is legitimate:

1. Declared Route Alignment: Does the claimed port call align with the vessel's declared trade route itinerary?
2. Timeline Context: Are neighboring records in the container's timeline clean and chronologically coherent?
3. Port Call Clustering Support: Do adjacent records from other containers confirm the vessel's presence at that port call?
4. Cryptographic Provenance Interlock: Does the record content hash match the committed block header on the provenance ledger?
5. Cargo Conservation Balance: Do incoming and outgoing mass and value state transitions balance across the call?

When arbitration identifies the legitimate record in a conflict pair, its carried evidence severity is multiplied by a damping factor γ ≤ 0.10 . This drops its fused probability below the suspicion threshold, assigning an ORIGINAL disposition. Conversely, the uncorroborated side retains its full severity s_i $\in [0.85, 1.00]$, driving its posterior probability above 0.95 for conviction.

Assess Empirical Blame Attribution Impact

The impact of conflict arbitration was evaluated across physical conflict codes:

Contradiction Finding Code	Pre-Arbitration Clean FP	Single-Tampered Pairs	Neither-Tampered Pairs	Post-Arbitration Clean FP
SIMULTANEOUS_PRESENCE	32	72	0	0
SPEED_INFEASIBLE	21	46	0	0
EVENT_ORDER_VIOLATION	3	3	0	0
Total Pipeline Aggregates	198	121	0	2

Pre-arbitration metrics show that every false positive on clean data was the innocent half of a genuine conflict pair (0 pairs involved two clean records). By resolving these contradictions through multi-signal corroboration, conflict arbitration reduced clean record false positives from 198 down to 2.

Section Transition

While conflict arbitration resolves pairwise record contradictions, cryptographic provenance provides immutable verification across historical event streams.

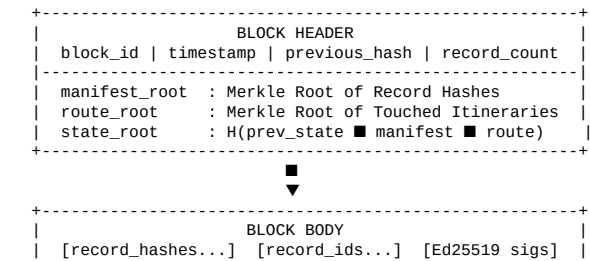
6. Provenance Blockchain & Distributed Virtual Consensus

Context & Strategic Mandate

To prevent retroactive database tampering, Makar incorporates an application-level permissioned provenance blockchain. By committing cryptographic hashes of records as events occur, the system creates a tamper-evident audit trail that detects unauthorized edits.

Analyze Merkle Tree Architecture & Block Structure

Makar's ledger stores structural cryptographic hashes rather than raw data values, protecting proprietary cargo details while proving record immutability.



+-----+

The core verification metric is the State Root, calculated as:

$$\text{state_root} = H(\text{previous_state_root} \parallel \text{manifest_root} \parallel \text{route_root})$$

The blockchain seals historical records up to a prefix cutoff of 85% of the operational timeline (`blockchain.sealed_fraction = 0.85`). The remaining 15% represents recent, unsealed events. This design forces the standalone forensic engines to evaluate recent records independently without relying solely on chain commitments.

Deconstruct Virtual Distributed Consensus & Node Corruption

The network operates using four independent virtual node processes: Node A, Node B, Node C, and Node D. These processes communicate using PyNaCl (Ed25519) signatures and SHA-256 assertions over local network sockets.

To test system resilience, Node C is configured as a compromised node (`nodes.compromised = [C]`). When an attacker alters historical records, Node C updates its local ledger blocks and recomputes its internal state roots to endorse the modified data.

```
Node A (Valid): State Root = 0x7f8a... (Agreed)
Node B (Valid): State Root = 0x7f8a... (Agreed)
Node D (Valid): State Root = 0x7f8a... (Agreed)
      ■
      ■■■> Majority Consensus State = 0x7f8a...
      ■
Node C (Malicious): State Root = 0x3b1c... ■■■ [ NODE_STATE_DIVERGENCE Detected ]
```

Although Node C's modified chain passes its local block hash checks, cross-node consensus checks immediately reveal state root divergence ($A=X, B=X, C=Y, D=X$). This isolates Node C and raises `NODE_STATE_DIVERGENCE` flags for all records endorsed exclusively by the rogue node.

Section Transition

Once tampered records are detected and verified cryptographically, the platform passes them to the reconstruction engine to recover their true historical states.

7. Candidate Reconstruction Engine & Field Accuracy

Context & Strategic Mandate

Identifying tampered records is only part of the forensic objective; system operators also require accurate reconstruction of original manifest parameters. Makar uses a candidate-based hypothesis generation engine that evaluates and scores proposed repairs against physical and contextual constraints, avoiding unvalidated overwrite operations.

Deconstruct Candidate Generation Strategies

When a record is classified as tampered, the reconstruction engine generates candidate original states using three hypothesis pathways:

- 1. **Adjacent Port Call Exploitation:** Exploits data redundancies across shipping legs. Because cargo state is shared across port call transitions, earlier or later records from the same port call often contain the original pre-tampered field values.
- 2. **Cargo Density & World Baselines:** Sources values from cargo-class distributions when adjacent record data is unavailable. Target attributes (such as mass or declared value) are derived using class-specific value-to-mass ratios.
- 3. **Record Removal (REMOVE RECORD):** Generates structural deletion hypotheses for fabricated records or redundant duplicates that lack valid lineage.

Synthesize Multi-Criterion Scoring Matrix

Candidate hypotheses are evaluated against a multi-criterion scoring matrix that measures consistency across five analytical dimensions:

$$\text{Score}(C_k) = w_{\text{route}} S_{\text{route}} + w_{\text{temp}} S_{\text{temp}} + w_{\text{cargo}} S_{\text{cargo}} + w_{\text{chain}} S_{\text{chain}}$$

Candidates must maintain route continuity, preserve temporal logic, balance cargo mass and value transitions, align with cryptographic hashes where available, and match established peer-group distributions. The candidate with the highest overall score above the repair threshold ($\text{Score}(C_k) \geq 0.85$) is selected as the reconstructed state. If no candidate meets the threshold, the record is classified as UNRECOVERABLE.

Analyze Empirical Field Reconstruction Accuracy

Field-level reconstruction performance was benchmarked across multi-seed evaluations:

Target Reconstruction Metric	Measured System Accuracy Range	Operational Analysis & Domain Mechanism
Overall Exact Record Repair	89.9% – 96.6%	Percentage of tampered records restored across all fields to their exact pre-attack values.
Destination Field Repair	100.0%	Reconstructed perfectly by leveraging declared route definitions and terminal port mapping.
Weight Field Repair	96.7%	High repair accuracy achieved by drawing target mass values from adjacent port call records.
Timestamp Field Repair	14.0% – 43.0%	Lower exact-point repair accuracy due to domain constraints governing temporal estimation.

Forensic Domain Commentary: Timestamp repair accuracy is lower because temporal neighbors define valid bounding intervals $[t_{\text{arrival}}, t_{\text{departure}}]$ rather than exact point values. While adjacent records share cargo mass (allowing exact point recovery), picking a single point timestamp within an 8-hour port window can introduce ungrounded data. Makar conservatively outputs verified bounding intervals rather than ungrounded timestamp point guesses, adhering to the principle that an acknowledged interval is superior to an unverified point estimate.

Section Transition

Behind these analytical and reconstruction engines is an interface suite that presents forensic findings through linked visual surfaces.

8. Comprehensive UI Suite & Forensic Investigation Interfaces

Context & Strategic Mandate

Makar provides an obsidian dark-mode interface suite designed for forensic investigations. The interface integrates relational graphs, geospatial trajectories, and temporal timelines into a unified workspace, allowing analysts to trace evidence seamlessly across multiple views.

- 1. Command Center:** High-level operational dashboard displaying real-time record disposition metrics (ORIGINAL, REPAIRED, REMOVED, UNRECOVERABLE), node network health monitors, and a priority queue of high-risk records.
- 2. Geo Forensic Map:** Built on MapLibre GL and Leaflet, this view renders spatial layers, trade corridors, vessel trajectories, speed anomalies, and interactive temporal playback controls to compare observed vs. reconstructed routes.
- 3. Bloodhound Graph:** A Cytoscape.js visual surface for exploring entity relationships. Features shortest-path evidence tracing, isolation filters for conflicting record pairs, and hub-aware 2-hop expansion logic. Naive graph expansion through major port entities (e.g., PORT_COL) causes node explosions (returning 1,995 nodes and 5,708 edges because a single port touches hundreds of shipments). Makar caps hub expansion to isolate the immediate 22-node structural context, preserving UI responsiveness and cognitive focus.
- 4. Attack Timeline Engine:** Dynamic temporal clustering view that groups individual anomalies into distinct temporal windows, helping analysts identify coordinated attack campaigns and campaign hypotheses.
- 5. Record Ledger:** Searchable workspace for filtering manifest records by disposition state, probability score, port, owner, and assigned tampering classification.
- 6. Record Investigation Workspace:** Deep-dive view rendering the complete step-by-step audit trail ($\text{Observation} \rightarrow \text{Evidence} \rightarrow \text{Inference} \rightarrow \text{Decision}$), displaying exact log-odds arithmetic and the conflict arbitration breakdown.
- 7. Reconstruction Workspace:** Displays all candidate repair hypotheses generated for a selected record alongside per-criterion scores and confidence metrics.
- 8. Provenance Monitor:** Distributed ledger inspector tracking permissioned block headers, sealed prefix boundaries, cryptographic Merkle roots, and live node consensus state roots (A=X, B=X, C=Y, D=X).

Section Transition

This visual interface suite supports both static batch evaluations and continuous, real-time live event feeds.

9. Live Stream Simulation & Novel Attack Patterns

Context & Strategic Mandate

In operational environments, manifests are continuously updated as events occur. Makar includes a live-streaming engine (generator/stream.py) to process real-time event feeds and evaluate emerging attack patterns without relying on static batch signatures.

Assess Real-Time Performance Metrics

The streaming engine was evaluated on a live feed containing 900 real-time events (48 injected attacks and 852 legitimate events). The system processed incoming records with an average latency of 1.6 ms per event, achieving a 0.00% false-alarm rate on untouched containers.

Deconstruct Novel Live Attack Patterns

To test adaptiveness, the live stream introduced three streaming attack patterns designed to bypass batch-oriented detection assumptions:

- 1. Weight Siphon:** Gradually reduces cargo weight across multiple consecutive transitions. Because each individual change remains within step-level tolerance thresholds, standard checks fail to flag the edit. Makar detects this using `CARGO_DRIFT_UNEXPLAINED`, which tracks multi-step cumulative drift. Crucially, `CARGO_DRIFT_UNEXPLAINED` fires only when every individual step is inside the per-step tolerance. Enforcing this boundary condition prevents duplicate findings and eliminates 14 false positives across test seeds.
- 2. Ghost Transfer:** Fabricates mid-voyage transfer events at transshipment hubs without prior arrival records, violating lineage constraints.
- 3. Identity Swap:** Exchanges container identifiers between adjacent shipments mid-voyage, maintaining individual record formatting while breaking historical continuity.

Analyze Streaming Evaluation Metrics

System performance across these live attack patterns was measured at both the individual event level and the container campaign level:

Novel Live Stream Attack Pattern	Defeated Operational Assumption	Event-Level Recall	Campaign-Level Recall
Ghost Transfer	Lineage Continuity	100.0%	100.0%
Identity Swap	Identity Continuity	25.0%	57.0%
Weight Siphon	Per-Step Tolerance Checks	26.0%	50.0%

Operational Callout: Event-level recall understates detection performance for extended attacks like siphoning or identity swaps. Once an initial state change occurs, subsequent events become internally consistent with the altered state. The critical operational metric is Campaign Recall—whether the system successfully detects and flags the targeted container during the attack window.

Section Transition

These streaming results complement Makar's broader batch performance, detailed in the following empirical benchmark and ablation studies.

10. Empirical Benchmarks & Ablation Study

Context & Strategic Mandate

To ensure statistical rigor, Makar was evaluated across multiple seed configurations. Parameters were tuned on an initial dataset (Seed 481516) and subsequently evaluated against two held-out seeds (Seed 271828 and Seed 999331) generated after parameter tuning was completed.

Synthesize Multi-Seed Benchmark Results

System performance across all three seed datasets is summarized below:

Measured Performance Metric	Seed 481516 (Tuned)	Seed 271828 (Held-Out)	Seed 999331 (Held-Out)	Aggregated Multi-Seed Performance
Precision	0.991	0.979	0.986	0.979 – 0.991
Recall	0.914*	0.925	0.851	0.851 – 0.925
F1-Score	0.951	0.951	0.914	0.914 – 0.951
Clean Record False Positives	2 / 4,522	5 / 4,537	3 / 4,532	2 – 5 records (< 0.11%)
Noisy Record False Positives	0 / 285	0 / 271	0 / 273	0 / 829 total (0.00%)
Deletion Recall	94.5%	98.2%	96.4%	94.5% – 98.2%
Exact Record Repair Accuracy	90.0%	96.6%	88.2%	88.2% – 96.6%

*Note: An initial tuning baseline yielded a recall of 0.910. The updated pipeline performance of 0.914 was achieved after resolving process-level set-iteration non-determinism during ID recycling in test_pipeline.py.

Across all three seeds, the system produced zero false alarms on noise-only records (0/829 total), confirming that formatting noise is effectively isolated from genuine security threats.

Deconstruct Detection Recall by Attack Class

Detection performance on the primary benchmark dataset (Seed 481516) broken down by specific attack category:

- Fabricated Records: 100.0% Recall (44 / 44 detected). Injected fabrications lack legitimate lineage and fail route/world consistency checks.
- Deleted Records (by slot): 94.5% Recall (52 / 55 detected). Identified by detecting missing expected event nodes within time-grouped sequences.
- Duplicated Records: 89.3% Recall (67 / 75 detected). Caught via time-windowed fuzzy payload matching and structural duplication checks.
- Modified Records: 89.6% Recall (112 / 125 detected). Uncovered through combined spatial, temporal, cargo conservation, and provenance checks.

Analyze System Ablation Metrics

To measure the contribution of the provenance layer relative to the standalone forensic engines, an ablation study was conducted with blockchain checks disabled:

Pipeline Configuration	Precision	Recall	F1-Score	Operational Interpretation
Full Architecture (All Engines Enabled)	0.991	0.914	0.951	Optimal configuration combining consistency checks with tamper-evident tracking.
Forensics Only (Blockchain Disabled)	0.978	0.369	0.536	High precision is maintained, but recall drops for subtle edits that break no physical rules.

The ablation demonstrates that while the blockchain layer significantly expands recall coverage by catching plausible field edits, the core forensic engines maintain high precision (0.978) when running independently.

Section Transition

The final section provides a step-by-step demonstration walkthrough showcasing these capabilities in an operational audit workflow.

11. 10-Step Interactive Demonstration Walkthrough

Context & Strategic Mandate

This operational walkthrough outlines a 10-step audit sequence for investigating compromised cargo manifests using the Makar platform interface.

Detail the 10 Execution Steps

Step 1: Ingest Suspect Manifest Data

Load the target suspect dataset (manifest_suspect.csv, containing 5,000 records) into the platform. Ingestion executes canonical normalization, raising 255 benign FORMAT evidence codes while structuring records for downstream analysis.

Step 2: Review Command Center Overview

Open the Command Center dashboard to inspect system disposition counters. The overview displays overall manifest status: 5,000 Total Records, 183 Suspicious Records, 71 Repaired Records, and 26 Removed Records, alongside node health monitors.

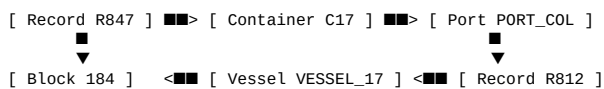
Step 3: Investigate High-Risk Record (Record R847)

Navigate to the Record Investigation Workspace and select record R847. The interface displays a 94.2% Tampering Probability alongside the explicit calculation breakdown:

```
L = -2.94 + \text{TEMPORAL}(2.002) + \text{SPATIAL}(1.740) + \text{ROUTE}(1.920) + \text{BLOCKCHAIN}(4.500) - \text{FORMAT}(0.000)
\text{Fused Probability } P = \frac{1}{1 + e^{-7.182}} = 0.942 \quad (94.2\%)
```

Step 4: Trace Evidence Paths in Bloodhound

Pivot to the Bloodhound Graph interface to trace relationships for record R847. Graph expansion reveals connected dependencies: Record R847 $\rightarrow$ Container C17 $\rightarrow$ Port PORT_COL $\rightarrow$ Vessel VESSEL_17 $\rightarrow$ Conflicting Record R812 $\rightarrow$ Block 184.



Step 5: Validate Spatial Anomaly on Geo Map

Open the Geo Forensic Map to inspect spatial trajectories for Container C17. The map highlights a speed anomaly on the transit leg between Mumbai (PORT_MUM) and Colombo (PORT_COL), requiring an impossible vessel speed of 91.4 knots (exceeding the vessel maximum of 28 knots).

Step 6: Identify Attack Windows

Open the Attack Timeline Engine to analyze temporal anomaly clustering. The engine groups active findings into a coordinated attack campaign between 02:14 and 02:31, pinpointing targeted operations involving PORT_SIN, OWNER_17, and VESSEL_42.

Step 7: Audit Provenance Ledger Consensus

Open the Provenance Monitor to evaluate multi-node state consistency. Cross-node comparisons isolate node divergence on Node C ($A=\text{0x7f8a}$, $B=\text{0x7f8a}$, $C=\text{0x3b1c}$, $D=\text{0x7f8a}$), identifying NODE_STATE_DIVERGENCE caused by local block manipulation.

Step 8: Execute Candidate Reconstruction

Navigate to the Reconstruction Workspace for record R847. The candidate engine evaluates proposed repairs, scoring Candidate A (18,200 kg, 96.8% confidence) above Candidate B (18,500 kg, 72.1% confidence), Candidate C (19,100 kg, 39.4% confidence), and Record Removal (18.2% confidence). Candidate A is selected and the record state is updated to REPAIRED.

Step 9: Replay Live Stream Simulation

Launch the live stream simulation view (scripts/stream_sim.py) to process 900 real-time events. Watch the incremental evidence engine catch streaming attack patterns (such as multi-step weight siphoning via CARGO_DRIFT_UNEXPLAINED) in real time with a 1.6 ms mean latency and zero false alarms on untouched containers.

Step 10: Export Auditable Forensic Report

Generate the final Suspicious Activity Report (out/report.md). The output synthesizes all findings, evidence chains, classification rationale, and candidate repair scores into an audit-ready forensic document suitable for court proceedings and trade compliance reviews.

Summary Architectural Conclusion

The Makar platform delivers an auditable, multi-layered forensic architecture for maritime manifest analysis. By decoupling raw observation from probability fusion, enforcing exculpatory noise treatment, resolving conflicting evidence through corroboration, and anchoring records with permissioned cryptographic chains, Makar achieves high detection precision while minimizing costly false alarms on legitimate trade cargo.